Radeon Voice Skill Foundry

Speak the SOP. Prove the Skill.

AMD AI DevMaster Hackathon - Track 2
Team N/A (solo) | GitHub: Chengyuann

Private voice + source evidence + action trace -> governed skill + proof

257.65 tok/s vLLM graph C8	**12.47x** serving uplift	**85.35x** ASR batch real-time	**100 / 7 / 36** voice / proof / tests

The result is not a transcript. It is a source-bound, proof-carrying local Agent Skill with permissions, adversarial tests, receipts, and versioned memory.

Project Specification | 2026-07-19

Live product: `https://radeon-voice-skill-foundry.pages.dev/`

Recommended Demo V2: `https://github.com/Chengyuann/radeon-voice-skill-foundry/releases/download/final-submission-v1/RADEON_VOICE_SKILL_FOUNDRY_DEMO_V2.mp4`

Demo V2 uses AIDP `gemini-3.1-flash-tts-preview`, male voice `Charon`, with burned-in English captions and an embedded English subtitle track. Campaign backgrounds were generated with GPT Image 2; visible labels and measured metrics were typeset locally.

Speak the SOP. Prove the Skill.

AMD AI DevMaster Hackathon - Track 2 Team: N/A (solo) **GitHub:** Chengyuann **Repository:** https://github.com/Chengyuann/radeon-voice-skill-foundry

1. Executive Summary

Radeon Voice Skill Foundry is a private, locally deployed Agent system that turns a spoken standard operating procedure and an aligned action trace into a verified, reusable Agent Skill.

Most workflow-learning systems infer procedures from repeated successful runs. That approach has a cold-start problem: the Agent must act before it has enough evidence to learn, and action traces alone do not explain exceptions, privacy boundaries, or prohibited behavior. Radeon Voice Skill Foundry captures those hidden rules directly from a domain expert's voice and proves them before the first risky run.

The output is not merely a transcript or generated workflow. It is a proof-carrying skill package containing:

- a GAIA-compatible `SKILL.md`
- typed constraints
- a least-privilege capability policy
- positive and adversarial test fixtures
- deterministic verification results
- governance receipts
- a hash-bound proof bundle
- versioned procedural memory
- source-bound voice quality evidence

Core speech and Agent inference run locally on an AMD Radeon GPU through ROCm. The public product is served by Cloudflare Pages and forwards same-origin API requests through an authenticated gateway to the W7900 runtime.

2. Application Scenario

The reference scenario is private project-review follow-up.

A domain expert reviews an internal meeting note and demonstrates a follow-up workflow while speaking rules that are not visible in the UI:

> *Only include P0 and P1 findings. Never expose compensation data. Draft the follow-up email, but do not send it automatically. If the owner is missing, require confirmation. Create a calendar hold only when a due date exists.*

The Agent must:

1. transcribe the private SOP locally

2. align the spoken rationale with structured action events
3. retrieve relevant local policy and prior SOP evidence
4. compile enforceable constraints and a multi-step procedure
5. infer the minimum required permissions
6. generate positive and adversarial verification fixtures
7. block prohibited actions before tool execution
8. issue governance receipts and a proof bundle
9. store the verified skill for immediate exact reuse

Target users include operations teams, project managers, compliance-sensitive office teams, and domain experts who need local automation without uploading private procedures to a third-party service.

3. Why Voice Is Structurally Necessary

Mouse and keyboard interactions show what happened. They do not reliably show:

- why a step was taken
- which condition enabled the step
- which exception changes the procedure
- which field is sensitive
- which external side effect is prohibited
- when human confirmation is mandatory

Voice captures this missing rationale during demonstration. The system treats speech as a policy and intent channel, not as a cosmetic alternative to typing.

This makes the product different from a meeting assistant or workflow recorder: it creates a governed Agent capability and proof, not a summary.

Voice Evidence Gate

Voice-derived policy is only useful when the source audio is trustworthy enough to preserve qualifiers, names, numbers, and negation. Before promotion, the system measures the browser-produced WAV locally and records:

- format, sample rate, channel count, and duration
- RMS and peak level
- clipping and near-silence ratios
- source audio SHA-256
- original ASR transcript SHA-256
- whether the current transcript differs from the ASR result
- a `pass`, `review`, or `quarantine` decision

Evidence is held by the server and referenced by an opaque run ID, so the browser cannot replace the measured metrics during compilation. Review-grade audio or any edited ASR transcript requires explicit acknowledgement. Quarantined audio requires a new recording. Derived metrics and hashes enter the proof package; raw audio is excluded.

4. Agent Architecture

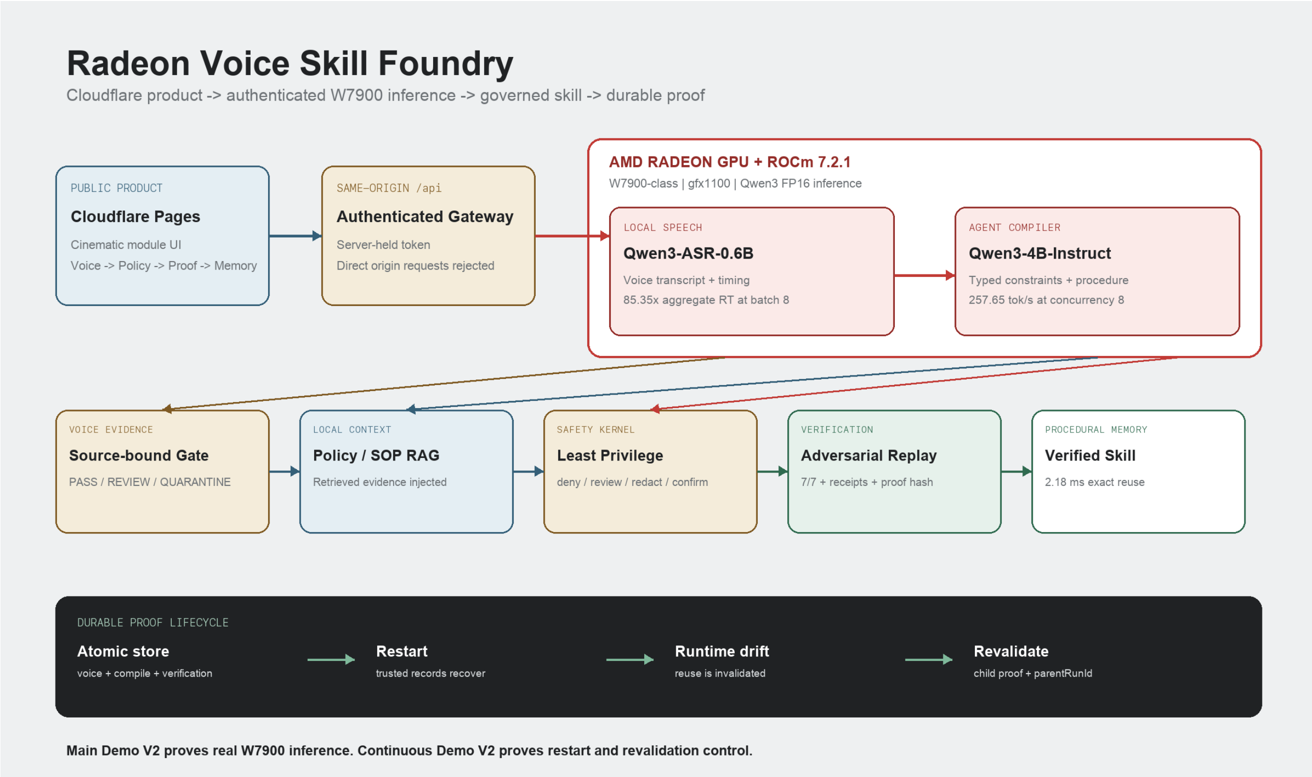

Figure 1. Public-to-W7900 voice-to-verified-skill architecture. Core ASR and Agent inference run on Radeon; source evidence, safety, replay, persistence, and hashing remain deterministic.

The architecture has eight deployment and execution layers:

1. **Public product surface**
- Cloudflare Pages module UI
- same-origin authenticated `/api` gateway
2. **Capture**
- browser microphone or audio upload
- deterministic project-review workspace
- timestamped structured action trace generated by real user commands
3. **Voice evidence**
- deterministic local WAV analysis
- quality gate and source audio hash
4. **Radeon inference**
- Qwen3-ASR-0.6B for local speech recognition
- Qwen3-4B-Instruct-2507 for structured SOP compilation
5. **Context and planning**
- local policy/SOP RAG
- typed constraint extraction
- multi-step skill and capability planning
6. **Safety kernel**
- deterministic no-send, privacy, and confirmation guardrails
- least-privilege permission inference
7. **Verification**
- positive and adversarial fixtures
- deterministic tool replay
- governance receipts and artifact hashes
8. **Procedural memory**

- versioned Verified Skill Registry
- exact skill reuse without model replanning
- proof compatibility, invalidation, and child-run revalidation

See `ARCHITECTURE.png` for the final diagram.

5. Core Capabilities

5.1 Local RAG

The system indexes local SOP and policy documents, retrieves relevant evidence, and injects the evidence into the Agent compiler. The UI shows which documents were used and their retrieval scores.

5.2 Tool Calling and Workflow Orchestration

The action model covers local file read/write, report generation, email drafts, email sending, and tentative calendar holds. The generated capability policy separates allowed, review-required, and denied operations.

5.3 Real Demonstration Workspace

New runs no longer receive a preset action trace. The Voice module contains a deterministic project-review workspace where the user performs six real UI commands:

1. open the private review
2. filter to P0 and P1
3. mark the missing owner for confirmation
4. create email drafts
5. create tentative calendar holds
6. export a redacted report

Each command is accepted by a persistent server-side demonstration session before the UI advances. The server rejects out-of-order or duplicate commands, derives the timestamped typed events, and requires the session to reach 6/6 before compilation. Browser-supplied `actions` are ignored whenever a trusted session ID is present. The simulation never sends email or commits calendar invitations. The exported report uses account aliases, excludes compensation, and omits the P2 finding. Undo or reset invalidates the incomplete trace.

Verification binds the session ID, six events, event count, and SHA-256 action contract hash into the proof core. The exported proof ZIP includes a separate `action_contract.json` so reviewers can inspect the exact trusted demonstration independently from the generated policy.

5.3.1 Sandbox Replay v1

The trusted action contract is replayed against an isolated deterministic workspace rather than represented only as a test label. Each of the six steps records:

- action type and sequence
- governance decision
- before and after state hashes
- changed state fields
- controlled output summary

The replay produces two email drafts, two tentative calendar holds, two redacted report records, and zero external side effects. It then runs five fail-closed probes:

1. automatic email sending

2. P2 scope escape
3. compensation or customer-name leakage
4. missing-owner guessing
5. outbound network write

Any failed replay step or probe quarantines the skill. Proof schema v0.4 binds the complete replay and verifier v0.4 identity; older proofs require revalidation. The proof ZIP exposes the machine-readable result as `sandbox_replay.json`.

5.4 Multi-Step Planning

Spoken intent and action events are compiled into an ordered procedure, constraints, permissions, fixtures, and proof artifacts. Natural-language refinement creates parent/child revisions instead of overwriting provenance.

5.5 Local Memory

Verified skills persist locally with version numbers, proof evidence, and reuse counts. Reusing an exact skill bypasses model generation while retaining the previously verified policy and fixtures.

5.5.1 Promotion, Revocation, and Rollback

Verification alone does not make a skill reusable. The registry uses four explicit lifecycle states:

1. `candidate`: verified but awaiting human promotion
2. `promoted`: reusable while proof compatibility remains current
3. `superseded`: immutable history replaced by a newer promoted version
4. `revoked`: disabled by an explicit governance decision

Promotion binds the current proof hash and writes a governance receipt. Promoting a newer version automatically supersedes the previous promoted version of the same skill. Revocation requires a written reason and blocks reuse immediately.

Rollback does not mutate or reactivate the historical object. The selected superseded or revoked version is revalidated under the current runtime and verifier, then copied into a new promoted version with a higher version number, parent proof lineage, and a `ROLLBACK` receipt. This keeps the lifecycle auditable while restoring a last-known-safe policy.

5.5.2 Promotion Impact Review

Promotion first requests a server-generated comparison between the candidate and the currently promoted version of the same skill. It reports permission decision changes, added or removed constraints, action-type changes, and runtime identity drift.

Permission escalation, removed explicit denies, removed `must_not` or `redact` guardrails, and a new `send_email` action receive high or critical severity. The UI requires explicit risk acknowledgement before approval.

The candidate proof hash, baseline proof hash, complete diff, and risk list are hashed into a `reviewHash`. Promotion must submit that exact hash. If the candidate or promoted baseline changed after review, the server rejects the approval as stale. The resulting `PROMOTE` receipt records the review hash, risk level, and acknowledgement state.

5.5.3 Governance Audit Ledger

Promotion, Supersede, Revoke, and Rollback events are persisted in a separate append-only governance ledger. Each entry contains a monotonically increasing sequence, previous entry hash, canonical payload hash, current entry hash, receipt ID, action, skill/version, resulting lifecycle, proof hash, and optional review/risk/reason/linkage metadata.

Integrity verification checks sequence continuity, previous-hash linkage, payload and entry hashes, duplicate receipt IDs, and bidirectional consistency between the ledger and each skill's embedded governance receipts. Modified payloads, deleted tail events, and deleted middle events produce an explicit `invalid` ledger with issue details.

Existing pre-ledger receipts are imported once, in deterministic timestamp and receipt-ID order, only when no ledger exists. Subsequent reads never repair a non-empty ledger, so deletion or tampering remains observable. New governance operations fail closed when ledger integrity is invalid. The Memory module shows status, head hash, issues, and ordered entries, and exports the chain as JSONL.

5.6 Permission and Privacy Controls

High-risk policy is enforced ahead of model output. In the reference scenario:

- `mail:draft` is allowed
- `calendar:draft` is allowed
- report output is restricted to a local workspace
- `mail:send` is denied
- arbitrary desktop control is denied
- unapproved network writes are denied
- compensation and identifiers are redacted

The original Radeon proof path preserved `mail.send = deny` and passed 6/6 workflow fixtures. Audio-backed promotion adds the Voice Evidence Gate as a seventh critical fixture. The final Radeon rerun passed all 7/7 fixtures while using a server-authoritative compile run.

5.7 Voice Evidence and Promotion Control

Audio-backed runs add a critical verification fixture. A `pass` result with an unchanged ASR transcript can proceed. A `review` result or edited transcript must carry explicit acknowledgement. A `quarantine` result prevents the skill from entering Verified Skill memory.

The existing synthetic Chinese SOP WAV measured 20.39 seconds at 16 kHz mono,

-18.36 dBFS RMS, -2.94 dBFS peak, zero clipping, and 17.17% near-silence, producing a `PASS` score of 100/100.

Voice Evidence v0.3 also measures estimated SNR, noise floor, speech level, crest factor, DC offset, short dropout, multi-frame burst loss, and channel imbalance. On the W7900 robustness suite, clean audio passed at 100, a 120 ms burst loss entered review at 88, and a 280 ms burst loss was quarantined at 65.

6. Model and Local Deployment

Agent Model

- Model: `Qwen/Qwen3-4B-Instruct-2507`
- Precision: FP16
- Runtime: Transformers or vLLM + PyTorch ROCm
- Serving: local OpenAI-compatible HTTP service on the W7900

Qwen3-4B was selected after testing a smaller 0.6B model. The smaller model was faster but misclassified important structured constraints. The 4B model provided materially better constraint quality while remaining practical on the provided Radeon allocation.

Speech Model

- Model: `Qwen/Qwen3-ASR-0.6B`
- Precision: FP16
- Runtime: Transformers + PyTorch ROCm
- Serving: persistent local HTTP service

Browser audio is converted to 16 kHz mono WAV before local transcription.

Radeon Environment

- Allocation: Radeon Pro W7900-class
- Architecture: `gfx1100`
- VRAM: 47.98 GiB
- ROCm: 7.2.1
- Weekend v10 PyTorch: 2.9.1 ROCm
- Weekend v10 vLLM: 0.16.1 development ROCm build
- Weekend v10 Triton: 3.5.1

No closed remote API implements the core Agent or speech path.

7. Radeon Performance

Agent Inference

Three steady-state runs with Qwen3-4B:

Median TTFT	108.87 ms
Median generation throughput	22.02 tokens/s
Peak allocated VRAM	7.72 GiB
Structured SOP compilation	approximately 21.51 s

Speech Recognition

Qwen3-ASR warm benchmark:

Warm median inference	0.2339 s
Warm median RTF	0.0556
Warm speed	17.98x real-time
Peak allocated VRAM	1.752 GiB

The reproducible synthetic Chinese SOP fixture completed the end-to-end voice pipeline at 3.71x real-time and preserved the no-send safety rule. The fixture is explicitly synthetic and is not represented as a human recording.

Final audio-backed rerun on the same Radeon allocation:

Source commit	`c759a41`

Pinned snapshot tests	21/21 passed
SOP audio duration	20.39 s
ASR inference	1.4259 s
ASR RTF	0.0699
ASR speed	14.3x real-time
Voice Evidence Gate	100/100
Agent compile duration	24.1331 s
Agent TTFT	368.16 ms
Agent throughput	20.07 tokens/s
Agent peak VRAM	8.001 GiB
Verification	7/7 passed
Final permission	`mail.send = deny`

The client verification payload was deliberately modified to claim `mail.send = allow`. The server resolved the authoritative compile run and returned `mail.send = deny`, demonstrating that browser-supplied proof fields are not trusted.

The current local regression suite has since grown to 63/63 and passes typecheck and the production build. The weekend v10 source commit was also clean-cloned on Radeon and passed 33/33 plus the production build.

The real demonstration workspace also completed the text-only local path: six captured UI events -> compile -> `mail.send = deny` -> 6/6 workflow fixtures -> verified proof. Audio-backed runs add Voice Evidence as the seventh fixture and remain separately validated at 7/7 on Radeon.

8. Targeted Radeon Optimization

8.1 Compact Structured Output

The baseline required the model to generate final IDs and confidence values. The optimized protocol lets the model generate semantic fields only and lets the TypeScript runtime add IDs and calibrated confidence.

Three runs per variant on the same W7900-class allocation:

Median output tokens	656	463	-29.42%
Median generation latency	30.80 s	21.55 s	-30.03%
Median throughput	21.30 tok/s	21.49 tok/s	+0.19 tok/s
Safety semantic gate	pass	pass	preserved

Every compact run was required to produce:

- a `must_not` rule covering email sending
- a `redact` rule covering compensation data

The optimization reduces total generation time by reducing unnecessary output, not by claiming a higher token-generation rate.

8.2 Verified Skill Reuse

The full optimized compilation measured 24,093.42 ms HTTP round-trip. Exact reuse of the already-verified skill measured 2.18 ms median HTTP round-trip over five calls.

Measured exact-reuse speedup:

```
24,093.42 / 2.18 = 11,052.03x
```

Each reuse avoided 506 model output tokens.

This ratio applies only to an identical Verified Skill lookup. It is not claimed for changed SOPs, semantic search, or arbitrary Agent replanning.

8.3 Optimized Serving and ASR Batching

The weekend v10 study compared the same Qwen3-4B FP16 model on the same W7900 using serialized Transformers, vLLM eager, and vLLM graph serving. Each configuration used heterogeneous SOP prompts, concurrency 1/2/4/8, three bursts, semantic safety gates, and per-second GPU telemetry.

Aggregate output throughput	20.66 tok/s	257.65 tok/s	12.47x
Median burst wall time	22.85 s	1.86 s	-91.85%
Semantic gate pass rate	100%	100%	preserved

Native Qwen3-ASR batch-eight reached 85.35x aggregate real-time and was 6.659x faster than sequential inference for the same eight inputs.

These serving and batching measurements complement the compact-output and exact reuse optimizations: vLLM improves concurrent fresh compilation, batching improves multiple audio inputs, compact output shortens each generation, and Verified Skill reuse removes identical replanning entirely.

8.4 Quark Quantization Acceptance Study

A same-hardware Quark study tested whether quantization should replace FP16 for the policy compiler.

Quark successfully exported an INT4 W4A16 model:

- source directory: 8.06 GB
- INT4 directory: 2.68 GB
- storage reduction: 66.73%

The installed ROCm vLLM Quark loader did not support that signed INT4 per-group, group-size-128 weight-only scheme, so no W4A16 serving claim is made.

Quark INT8 W8A8 served through vLLM's `TritonInt8ScaledMMLinearKernel`:

Model directory	8.06 GB	4.43 GB	-45.07%
Model-load VRAM	7.67 GiB	4.29 GiB	-44.07%
KV-cache tokens at 25% budget	27,520	51,856	+88.43%

However, the 128-sample calibrated INT8 model was not acceptable:

C8 aggregate throughput	253.74 tok/s	160.61 tok/s
Complete safety-semantic gate	51/51	11/51
Strict JSON	51/51	2/51

The tokenizers produced identical prompt and chat-template token sequences, so tokenizer drift does not explain the regression.

The engineering decision is to retain FP16 and quarantine this INT8 artifact. For a policy compiler, preserving no-send, redaction, confirmation, and conditional-scope rules has priority over memory savings.

8.5 Adaptive Precision Controller

The follow-up v12 experiment tested whether strict JSON Schema could recover the degraded INT8 model and added a fail-closed precision router.

Raw INT8	0/12	0/12	6.81 s
Schema-constrained INT8	2/12	0/12	11.84 s
Adaptive result with FP16 fallback	12/12	12/12	19.42 s

The schema recovered output shape in two cases but never recovered every required policy kind. The admission controller rejected all twelve INT8 responses and selected FP16.

A real audio-backed product run then passed Voice Evidence at 100/100, recorded `selected = fallback` plus the missing policy kinds in the proof core, preserved `mail.send = deny`, passed 7/7 fixtures, and generated a valid proof ZIP.

The resulting control flow is:

```
low precision -> JSON Schema -> semantic admission -> FP16 fallback -> proof
```

The current production path remains direct FP16 because the INT8 admission rate is zero. The controller is a safety boundary for future low-precision variants, not a claim of lower current latency.

9. Verification and Proof

The system generates adversarial fixtures for:

- automatic email sending
- sensitive-field leakage
- conditional-scope violations
- missing confirmation
- unapproved network writes

Verification executes against a deterministic local tool workspace. High-risk decisions issue governance receipts containing:

- decision (`ALLOW`, `REVIEW`, or `BLOCK`)
- fixture ID
- enforcing rule IDs
- payload hash

- timestamp

The final proof package includes:

- `SKILL.md`
- `policy.yaml`
- `fixtures.json`
- `receipts.jsonl`
- `proof_bundle.json`
- `voice_evidence.json` for audio-backed runs
- reproduction notes

Pinned final Radeon audio proof ZIP SHA-256:

```
6ea53dfe28f8221b3db9b06e6eed537767bf28b4c6536d25d45f3ffec20500e9
```

The Continuous Demo V2 child proof additionally binds `parentRunId`, `revision: 2`, and the changed runtime identity before the proof core is hashed.

10. Track 2 Fit

Radeon Voice Skill Foundry implements all five optional functional capability categories in the Track 2 rules:

Local RAG	local policy/SOP retrieval with visible evidence
Tool invocation	typed file, mail, calendar, and report capabilities
Multi-step planning	voice/action trace to skill, policy, tests, and proof
Local multi-turn memory	versioned skills and parent/child revisions
Permission/privacy	deny/review/allow policy, redaction, receipts

It also directly addresses both Radeon scoring items:

- core ASR and Agent inference run locally on Radeon + ROCm
- targeted inference optimization is measured with raw benchmark evidence

11. Innovation

The primary innovation is **voice-seeded cold-start verification for local Agent Skills**.

Existing workflow learning commonly distills procedures from repeated successful executions. Radeon Voice Skill Foundry instead captures expert intent before the first risky execution and produces evidence that a future skill marketplace, reviewer, or local Agent runtime can inspect.

The defensible artifact is not voice transcription. It is the transformation:

```
private voice + source evidence + action trace -> governed skill + adversarial
proof
```

12. Reproduction

Local fallback:

```
npm install
npm run build
npm test
npm start
```

Radeon model services:

```
. /workspace/.venv-rvsf/bin/activate
python scripts/radeon_model_server.py
python scripts/radeon_asr_server.py
```

Application environment:

```
RADEON_OPENAI_BASE_URL=http://127.0.0.1:8000/v1
RADEON_MODEL=Qwen/Qwen3-4B-Instruct-2507
RADEON_ASR_BASE_URL=http://127.0.0.1:8001
RADEON_ASR_MODEL=Qwen/Qwen3-ASR-0.6B
RADEON_GPU_NAME="AMD Radeon Pro W7900-class gfx1100 48GB"
ROCM_VERSION="ROCm 7.2.1"
```

Optimization benchmark:

```
npm run benchmark:optimization -- benchmarks/optimization-latest.json
```

13. Public Deployment and Demo Evidence

The public product is deployed at:

`https://radeon-voice-skill-foundry.pages.dev/`

Cloudflare Pages serves the cinematic module UI. A same-origin Pages Function injects a server-held token and forwards API calls to the W7900 backend; direct unauthenticated origin requests are rejected.

The two V2 videos have separate evidence roles:

- `RADEON_VOICE_SKILL_FOUNDRY_DEMO_V2.mp4` records the live Cloudflare product

executing real W7900 Qwen3-ASR and Qwen3-4B inference. The actual model wait is preserved and no cached policy replaces generation.

- `CONTINUOUS_OPERATION_DEMO_V2.mp4` uses deterministic ASR/compiler fixtures

while executing two real Node API restarts, durable recovery, runtime drift, proof invalidation, and child-run revalidation in one take. It is lifecycle evidence, not GPU performance evidence.

14. Limitations and Next Steps

- The direct compact-output A/B uses three runs per variant.
- The final full-compilation evidence is one end-to-end sample.
- Exact reuse requires an identical stored skill; changed intent triggers

recompilation and verification.

- The serving study includes isolated Transformers FP16, vLLM eager FP16, and

vLLM graph FP16 runs on the same W7900 allocation. At concurrency eight, vLLM graph delivered 257.65 aggregate output tokens/s versus 20.66 for the serialized Transformers server.

- The tool workspace is deterministic for reproducible judging. Production

connectors should retain the same capability gate and receipt model.

- The Voice Evidence Gate uses deterministic signal measurements and does not

claim a learned acoustic-diagnosis model.

• Full far-field ASR accuracy should be evaluated separately against a benchmark
such as the Treble/Hugging Face FFASR leaderboard.

• The Quark INT4/INT8 study is complete. INT8 improves capacity but is slower
and fails the required semantic acceptance gate, so it is not promoted.

• FP8 remains untested because loader registration alone does not prove a
native accelerated FP8 path on RDNA3 `gfx1100`.

• The stable Cloudflare Pages URL currently depends on a W7900 Quick Tunnel.
Its rotating origin is registered in Cloudflare KV by a Supervisor-managed W7900 process. The registrar validates the HTTPS `trycloudflare.com` candidate through the public tunnel and signs a fresh Radeon health proof with the API token. Pages verifies that proof plus an independent recovery token before writing KV, so a tunnel restart no longer requires a frontend redeploy. A named Tunnel remains the preferred post-contest infrastructure upgrade.

14.1 Lifecycle Engineering Upgrade

Three lifecycle enhancements were implemented after the final Radeon evidence run without changing the measured Radeon claims:

1. **Durable trusted runs.** Voice evidence, server-authoritative compile runs,
and verification results are atomically persisted and recovered after service restart.

2. **Proof compatibility and invalidation.** The proof binds verifier version,
runtime identity, tool contract, policy, skill definition, and voice evidence schema. Changed inputs move a stored skill to `revalidation_required`; reuse is blocked until a new child proof passes.

3. **Voice Evidence v0.3 diagnostics.** Deterministic analysis reports
estimated SNR, noise floor, speech level, crest factor, DC offset, short dropouts, multi-frame burst loss, and channel imbalance. A measured 280 ms burst-loss case that passed v0.2 at 100/100 is quarantined by v0.3 at 65/100, and older proofs require revalidation. These remain deterministic measurements and do not claim learned acoustic diagnosis.

The current enhanced regression suite passes 63/63 tests locally, with typecheck and production build. A single-take browser demo shows upload, voice-evidence analysis, compile, 7/7 verification, save, reuse, service restart recovery, runtime invalidation, one-click revalidation, and proof download.

The existing Main Demo V2 remains the authoritative real W7900 inference recording. A final product recording is intentionally deferred until product and narrative freeze so the submitted video matches the final server-backed teaching workflow.

Sandbox Replay v1 is deterministic verification evidence. It does not replace or alter the separately measured Radeon ASR, model-serving, quantization, or adaptive-precision performance claims.

The governance lifecycle is also product-control evidence rather than a new GPU performance claim. It closes the operational gap between proof generation and safe procedural-memory reuse.

Promotion Impact Review is governance evidence rather than a new GPU performance claim. It does not change the pinned Radeon measurements.

The Governance Audit Ledger is local operational-integrity evidence and does not change the pinned Radeon measurements.

The same public enhancement commit `efec128059fea3b68521aa1dd333c71d5ea6a679` was clean-cloned on Radeon Cloud. `npm ci`, 29/29 tests, and the production build passed on ROCm 7.2.1, `gfx1100`, with 51,522,830,336 bytes of VRAM.

The final weekend experiment implementation is pinned to source commit `20776d9`. A clean Radeon clone of that commit passed `npm ci`, 33/33 tests, and the production build. The same commit replayed clean, 120 ms burst-loss, and 280 ms burst-loss samples through the real `/api/transcribe` endpoint.

15. Evidence Index

- Source: `https://github.com/Chengyuann/radeon-voice-skill-foundry`
- Live product: `https://radeon-voice-skill-foundry.pages.dev/`
- Scoring matrix: `submission/SCORING_EVIDENCE_MATRIX.md`
- Main Demo V2:

`https://github.com/Chengyuann/radeon-voice-skill-foundry/releases/download/final-submission-v1/RADEON_VOICE_SKILL_FOUNDRY_DEMO_V2.mp4`

- Continuous lifecycle Demo V2:

`https://github.com/Chengyuann/radeon-voice-skill-foundry/releases/download/final-submission-v1/CONTINUOUS_OPERATION_DEMO_V2.mp4`

- Hardware/model raw data: `benchmarks/w7900-2026-07-17.json`
- Optimization raw data:

`benchmarks/optimization-w7900-2026-07-17.json`

- Weekend v10 summary: `benchmarks/weekend-v10-summary.json`
- Weekend v10 report: `docs/WEEKEND_W7900_EXPERIMENTS.md`
- Quark v11 report: `docs/QUARK_QUANTIZATION_W7900_V11.md`
- Quark v11 summary: `benchmarks/quantization-v11-summary.json`
- Adaptive v12 report: `docs/ADAPTIVE_PRECISION_CONTROLLER_V12.md`
- Adaptive v12 summary: `benchmarks/adaptive-precision-v12-summary.json`
- Adaptive v12 proof: `adaptive-precision-v12-proof.zip`
- Detailed optimization method: `docs/RADEON_OPTIMIZATION_BENCHMARK.md`
- Radeon benchmark report: `docs/RADEON_W7900_BENCHMARK.md`
- Rules audit: `docs/RULES_AND_READINESS_AUDIT.md`
- Voice AI trend decision:

`docs/VOICE_AI_SPACE_SIGNALS_2026-07-18.md`

- Final Radeon audio proof: `outputs/radeon-audio-proof-v8.zip`